UNIT -3

KNOWLEDGE REPRESENTATION

Knowledge based agent

Knowledge-based agents have explicit representation of knowledge that can be reasoned. They maintain internal state of knowledge, reason over it, update it and perform actions accordingly. These agents act intelligently according to requirements.
Knowledge based agents give the current situation in the form of sentences. They have complete knowledge of current situation of mini-world and its surroundings. These agents manipulate knowledge to infer new things at "Knowledge level".

Why use a knowledge base?

- A knowledge base **inference** is required for updating knowledge for an agent to learn with experiences and take action as per the knowledge.
- Inference means deriving new sentences from old. The inference-**based** system allows us to add a new sentence to the knowledge base. A sentence is a proposition about the world. The inference system applies logical rules to the KB to deduce new information.
- **The inference** system generates new facts so that an agent can update the KB. An inference system works mainly in two rules which are given:
    - Forward chaining
    - Backward chaining

Various levels of knowledge-based agents

A knowledge-based agent can be viewed at different levels which are given below:

1. Knowledge level
Knowledge level is the first level of knowledge-based agent, and in this level, we need to specify what the agent knows, and what the agent goals are. With these specifications, we can fix its behavior. For example, suppose an automated taxi agent needs to go from a station A to station B, and he knows the way from A to B, so this comes at the knowledge level.

2. Logical level
At this level, we understand that how the knowledge representation of knowledge is stored. At this level, sentences are encoded into different logics. At the logical level, an encoding of knowledge into logical sentences occurs. At the logical level we can expect to the automated taxi agent to reach to the destination B.

3. Implementation level
This is the physical representation of logic and knowledge. At the implementation level agent perform actions as per logical and knowledge level. At this level, an automated taxi agent actually implement his knowledge and logic so that he can reach to the destination.

knowledge-based system has following features

Knowledge base (KB): *It is the key component of a knowledge-based agent. These deal with real facts of world. It is a mixture of sentences which are explained in knowledge representation language.*

Inference Engine(IE): It is knowledge-based system engine used to infer new knowledge in the system.

Actions performed by an agent

Inference System is used when we want to update some information (sentences) in Knowledge-Based System and to know the already present information. This mechanism is done by **TELL** and **ASK** operations. They include inference i.e. producing new sentences from old. Inference must accept needs when one asks a question to KB and answer should follow from what has been Told to KB. Agent also has a KB, which initially has some background Knowledge. Whenever, agent program is called, it performs some actions.

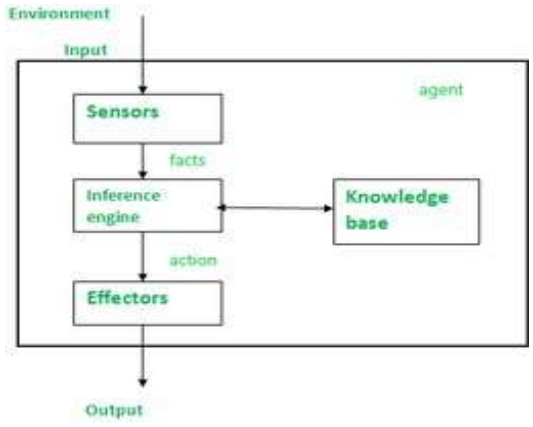
Actions done by KB Agent:

1. It TELLS what it recognized from the environment and what it needs to know to the knowledge base.
2. It ASKS what actions to do? and gets answers from the knowledge base.
3. It TELLS the which action is selected , then agent will execute that action.

Algorithm :

function KB_AGENT (percept) returns an action
KB : knowledge base
t : time (counter initially 0)
TELL(KB, MAKE_PERCEPT_SENTENCE (percept,t))
action = ASK(KB, MAKE_ACTION_QUERY (t))
TELL(KB, MAKE_ACTION_SENTENCE (action,t))
t = t + 1
return action

If a percept is given, agent adds it to KB, then it will ask KB for the best action and then tells KB that it has in fact taken that action.



A Knowledge based system behavior can be designed in following approaches:-

Declarative Approach*:* In this beginning from an empty knowledge base, the agent can TELL sentences one after another till the agent has knowledge of how to work with its environment. This

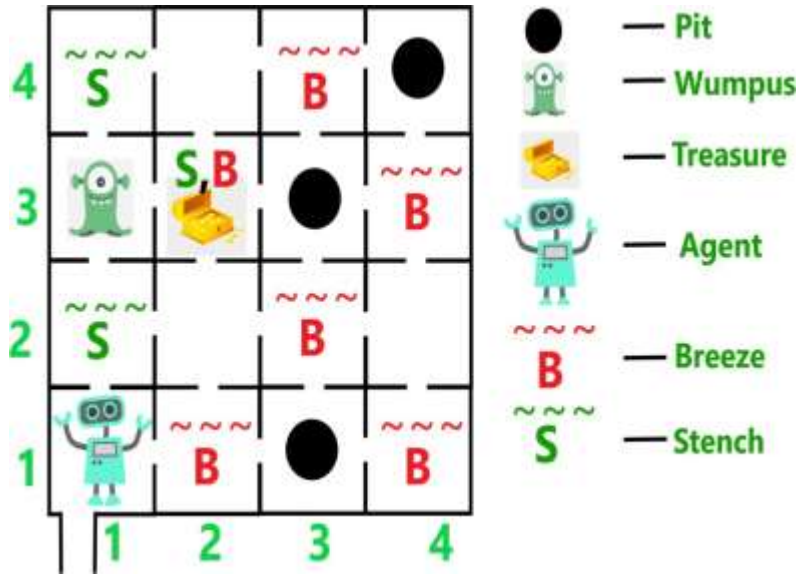
is known as the declarative approach. It stores required information in empty knowledge-based system.
Procedural Approach*:* This converts required behaviors directly into program code in empty knowledge-based system. It is a contrast approach when compared to Declarative approach. In this by coding behavior of system is designed .

The Wumpus world

The Wumpus World's agent is an example of a knowledge-based agent that represents Knowledge representation, reasoning and planning. **Knowledge-Based agent** links general knowledge with current percepts to infer hidden characters of current state before selecting actions. Its necessity is vital in partially observable environments.

Problem Statement:

The Wumpus world is a cave with *16* rooms *(4×4)*. Each room is connected to others through walkways (no rooms are connected diagonally). The knowledge-based agent starts from *Room[1, 1]*. The cave has – some **pits**, a **treasure** and a beast named **Wumpus**. The Wumpus can not move but eats the one who enters its room. If the agent enters the pit, it gets stuck there. The goal of the agent is to take the treasure and come out of the cave. The agent is rewarded, when the goal conditions are met. The agent is penalized, when it falls into a pit or being eaten by the Wumpus. Some elements support the agent to explore the cave, like -The wumpus's adjacent rooms are stenchy. -The agent is given one arrow which it can use to kill the wumpus when facing it (Wumpus screams when it is killed). – The adjacent rooms of the room with pits are filled with breeze. -The treasure room is always glittery.



PEAS Description for the Wumpus World problem:

1. **Performance measures:**

- Agent gets the gold and return back safe = *+1000 points*

- Agent dies = *-1000 points*
- Each move of the agent = *-1 point*
- Agent uses the arrow = *-10 points*

2. **Environment:**
- A cave with *16(4×4)* rooms
- Rooms adjacent (not diagonally) to the Wumpus are stinking
- Rooms adjacent (not diagonally) to the pit are breezy
- The room with the gold glitters
- Agent's initial position – *Room[1, 1]* and facing right side
- Location of Wumpus, gold and *3 pits* can be anywhere, except in *Room[1, 1]*.

3. **Actuators:**

Devices that allow the agent to perform the following actions in the environment.
- Move forward
- Turn right
- Turn left
- Shoot
- Grab
- Release

4. **Sensors:**

   Devices which helps the agent in sensing the following from the environment.
   - Breeze
   - Stench
   - Glitter
   - Scream (When the Wumpus is killed)
   - Bump (when the agent hits a wall)

Wumpus World Characterization:

- **Partially Observable:** knows only the local perceptions
- **Deterministic:** outcome is precisely specified
- **Sequential:** subsequent level of actions performed
- **Static:** Wumpus, pits are immobile
- **Discrete:** discrete environment
- **Single-agent:** The knowledge-based agent is the only agent whereas the wumpus is considered as the environment's feature.

Propositional logic

Propositional logic represents knowledge/ information in terms of propositions. Prepositions are facts and non-facts that can be true or false. Propositions are expressed using ordinary declarative sentences. Propositional logic is the simplest logic.

Syntax:

The syntax of propositional logic defines the allowable sentences. The atomic sentences- the indivisible syntactic elements- consist of single proposition symbol. Each such symbol stands for a proposition that can be true or false. We use the symbols like P1, P2 to represent sentences.

The complex sentences are constructed from simpler sentences using logical connectives. There

are five connectives in common use:

¬ (*negation*), ^ (*conjunction*), ∨ (*disjunction*), ⇒ (*implication*), ⇔ (*biconditional*) The order of precedence in propositional logic is from (highest to lowest): ¬ , ^ , ∨, ⇒, ⇔.

Propositional logic is defined as:

If S is a sentence, ¬S is a sentence (*negation*)

If S1 and S2 are sentences, S1 ^ S2 is a sentence (*conjunction*)

If S1 and S2 are sentences, S1 ∨ S2 is a sentence (*disjunction*)

If S1 and S2 are sentences, S1 ⇒ S2 is a sentence (*implication*)

If S1 and S2 are sentences, S1 ⇔ S2 is a sentence (*biconditional*)

Formal grammar for propositional logic can be given as below:

Sentence	→ AutomicSentence | ComplexSentence
AutomicSentence	→ True | False | Symbol
Symbol	→ P | Q | R ……
ComplexSentence	→ ¬Sentence
	| (Sentence ^ Sentence)
	| (Sentence ∨ Sentence)
	| (Sentence ⇒ Sentence)
	| (Sentence ⇔ Sentence)

Semantics:

Each model specifies true/false for each proposition symbol

Rules for evaluating truth with respect to a model:

¬S is true if, S is false

S1 ^ S2 is true if, S1 is true and S2 is true

S1 ∨ S2 is true if, S1 is true or S2 is true

S1 ⇒ S2 is true if, S 1 is false or S2 is true

S1 ⇔ S2 is true if, S1 ⇒ S2 is true and S2 ⇒ S1 is true

Truth Table showing the evaluation of semantics of complex sentences:

P	Q	¬P	P∧Q	P∨Q	P⇒Q	P⇔Q
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Explain the Key Components of Propositional Logic:

1. **Propositions (Statements):** In propositional logic, propositions are the basic building blocks. A proposition represents a declarative statement that can be either true or false but not both. These statements can be about anything, from simple facts to more complex statements. For example, "The sky is blue" is a proposition because it can be evaluated as either true or false.

2. **Logical Operators:**

AND (Conjunction): The AND operator, often represented by ∧, is used to combine two or more propositions. It yields a true result only when all the propositions it connects are true. For example, if P represents "It's raining" and Q represents "I have an umbrella," then P ∧ Q is true only if both "It's raining" and "I have an umbrella" are true.

OR (Disjunction): The OR operator, represented by ∨, combines propositions and results in true if at least one of the connected propositions is true. For instance, if P represents "It's sunny" and Q represents "It's warm," then P ∨ Q is true if either "It's sunny" or "It's warm" or both are true.

NOT (Negation): The NOT operator, represented by ¬, is used to negate or reverse the truth value of a single proposition. If P is "The store is open," then ¬P is true if the store is not open.

3. Truth Values (True and False): Every proposition in propositional logic has one of two possible truth values—true or false. These truth values indicate whether the statement is valid or not in a given context.

Syntax of Propositional Logic in Artificial Intelligence

In propositional logic, the syntax defines how propositions are constructed. The key components include:

1. Variables: Variables in propositional logic represent unknown or unspecified statements. They are typically denoted by single uppercase letters, such as P, Q, or R. These variables can stand for any proposition.

2. Constants: Constants are specific, unchanging propositions that are either true or false. They are often represented using lowercase letters or other symbols. For example, 'p' might represent the constant "It's raining," which is either true or false.

3. Connectives (Logical Operators): Connectives are symbols that are used to combine propositions and create compound propositions. The main connectives in propositional logic include:

- Conjunction (AND, ∧): Denoted by ∧, this connects two propositions and results in a compound proposition that is true only if both of its components are true.
- Disjunction (OR, ∨): Denoted by ∨, this connects two propositions and results in a compound proposition that is true if at least one of its components is true.
- Negation (NOT, ¬): Denoted by ¬, this applies to a single proposition and negates it, reversing its truth value.

Propositional theorem proving

This article discusses how to use inference rules to create proof—a series of conclusions that leads to the desired result. The most well-known rule is known as **Modus Ponens** (Latin for affirming mode) and is expressed as

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

Inferences in Proving Propositional Theorem

The notation signifies that the sentence may be deduced whenever any sentences of the type are supplied.

If $(\text{WumpusAhead} \wedge \text{WumpusAlive}) \Rightarrow \text{Shoot}$ and $(\text{WumpusAhead} \wedge \text{WumpusAlive})$ are both supplied, Shoot may be deduced.

And-Elimination is another helpful inference rule, which states that any of the conjuncts can be inferred from conjunction:

$$\frac{\alpha \wedge \beta}{\alpha}$$

WumpusAlive can be deduced from $(\text{WumpusAhead} \wedge \text{WumpusAlive})$, for example. One may readily demonstrate that Modus Ponens and And-Elimination are sound once and for all by evaluating the potential truth values of and . These principles may then be applied to each situation in which they apply, resulting in good conclusions without the necessity of enumerating models.

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$
$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$
$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$
$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$
$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$
$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$
$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{De Morgan}$$
$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{De Morgan}$$
$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$
$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

The equations above show all of the logical equivalences that can be utilized as inference rules. The equivalence for biconditional elimination, for example, produces the two inference rules.

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \quad \text{and} \quad \frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

Some inference rules do not function in both directions in the same way. We can't, for example, run Modus Ponens in the reverse direction to get $\alpha \Rightarrow \beta$ and α from β

Let's look at how these equivalences and inference rules may be applied in the wumpus environment. We begin with the knowledge base including R1 through R5 and demonstrate how to establish $\neg P_{1,2}$ i.e. that [1,2] does not include any pits. To generate R6, we first apply biconditional elimination to R2:

$$R_6: \quad (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

After that, we apply And-Elimination on R6 to get $R_7: \quad ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$

For contrapositives, logical equivalence yields $R_8: \quad (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}))$

With R8 and the percept R_4 (i.e., $\neg B_{1,1}$), we can now apply Modus Ponens to get $R_9: \quad \neg(P_{1,2} \vee P_{2,1})$

Finally, we use De Morgan's rule to arrive at the following conclusion: $R_{10}: \quad \neg P_{1,2} \wedge \neg P_{2,1}$

That is to say, neither [1,2] nor [2,1] have a pit in them.

We found this proof by hand, but any of the search techniques may be used to produce a proof-like sequence of steps. All we have to do now is define a proof problem:

- **Initial State:** the starting point for knowledge.
- **Actions:** the set of actions is made up of all the inference rules that have been applied to all the sentences that fit the inference rule's upper half.
- **Consequences:** Adding the statement to the bottom part of the inference rule is the result of an action.
- **Objective:** The objective is to arrive at a state that contains the phrase we are attempting to verify.

As a result, looking for proofs is a viable alternative to counting models.

Agent based on the propositional logic

- An AI agent that utilises propositional logic to express its knowledge and make decisions is known as a propositional logic-based agent.
- A straightforward form of agent, it decides what to do depending on what it knows about the outside world.
- A knowledge base, which is made up of a collection of logical phrases or sentences, serves as a representation of the propositional logic-based agent's knowledge.
- The agent's knowledge is empty, however as it observes the outside world, it fills it with fresh data. To decide what actions to do in response to the environment, the agent uses its knowledge base.
- Depending on the logical inference it makes on its knowledge base, the agent takes judgements.

How does a Propositional Logic-based Agent work?

A propositional logic-based agent functions by expressing its understanding of the outside world as logical statements. The knowledge base is initially empty, but as the agent explores the environment, it fills it with fresh data. The agent draws new knowledge from its knowledge base through logical inference. Deductive or inductive reasoning can be used to draw a conclusion**.**

Deductive inference is the process of inferring new information using logical principles from already known information. The process of generalizing from specific data to arrive at a broader conclusion is known as inductive inference. Based on the objectives it seeks to attain, the agent decides what course of action to take.

Perception, reasoning, and action are the three stages of the agent's decision-making process. Observing the surroundings and updating the information base are steps in the perception process. In order to generate new information, the reasoning stage requires using logical inference to the knowledge base. The action phase entails choosing an action based on the information that was gathered and the agent's objectives.

Applications of Propositional Logic-based Agents

In the field of AI, propositional logic-based agents have several uses. Expert system applications are one of the most popular uses. Expert systems are artificial intelligence programs created to address difficulties in a particular field. They represent their subject knowledge in a knowledge base, and they draw new information from the knowledge base using a reasoning engine.

1) **Knowledge Representation**

- Propositional logic-based agents' core feature is knowledge representation.
- A collection of logical clauses that represent the agent's knowledge of the outside world make up the knowledge base of the agent.
- Depending on how much knowledge the agent has about the outside world, the knowledge base may be either complete or lacking.
- The agent's capacity to make informed decisions is impacted by the knowledge base's completeness.
- The fact that propositional logic offers a straightforward and understandable method of conveying knowledge is one of its benefits.
- Propositional logic uses simple to comprehend logical symbols and logical connectives to depict relationships between propositions.

2) Logical Inference

- The technique of inferring new knowledge from knowledge already known is known as logical inference.
- Propositional logic-based agents should have logical inference because it enables the agent to reason regarding the external world and gather new knowledge that can be applied to decision-making.
- Deductive inference as well as inductive inference are the two different categories of logical inference.
- By using logical principles, deductive inference is the act of obtaining new knowledge based on previously data obtained.
- It is predicated on the idea that if an argument's premises are true, then it follows that the argument's conclusion must also be true. Propositional logic-based agents draw new knowledge from the body of knowledge through deductive inference.

3) Decision Making

- A crucial function of propositional logic-based agents is decision-making.
- The agent bases its decisions on knowledge of the outside world and its desired outcomes.
- Three steps make up the decision-making process: perception, justification, and execution.
- Observing the environment and updating the agent's knowledge base is the process of perception.
- Using logical inference to extract new information from the knowledge base is the process of reasoning.
- Action is the process of choosing a course of action based on the knowledge that has been obtained and the agent's goals.
- Making judgements in a transparent and understandable manner is one of the advantages of employing propositional logic-based agents for decision making.
- It is simpler to trust the agent's conclusions since the logical rules it uses to decide are simple enough for humans to understand.

4) Limitations

- Although agents based on propositional logic offer numerous benefits, they also have certain drawbacks.
- One of the drawbacks is that they lack expressiveness and are unable to depict intricate interactions between propositions.
- They are unable to depict, for instance, causal or temporal links between assertions.

- Another drawback is that propositional logic-based agents are unable to deal with uncertainty or inadequate data.
- As a result, they are unable to handle circumstances in which there is a lack of information or uncertainty regarding the environment.

5) Conclusion

- In conclusion, Propositional rationale-based specialists give a primary simulated intelligence strategy to data portrayal and direction.
- They can be used in a variety of ways and can be combined with other AI models to make better systems.
- They are useful in industries that require trust and transparency, despite their limitations. Propositional logic-based agents will continue to be crucial to the development of intelligent systems as AI research advances.
- Where information can be expressed as propositions and decision-making follows logical principles, their effectiveness shines especially brightly.

First-Order logic syntax and semantics of First-Order logic

First Order Logic in Artificial Intelligence is a technique used for **knowledge representation**. It is an extension of propositional logic and unlike propositional logic, it is sufficiently expressive in representing any natural language construct. First Order Logic in AI is also known as **Predicate Logic** or **First Order Predicate Logic**. It is a robust technique to represent objects as well as their relationships. Unlike propositional logic, First Order Logic in Artificial Intelligence doesn't only include facts but also different other entities as listed below.

1. **Objects:**
   Objects can denote any real-world entity or any variable.
   E.g., A, B, colours, theories, circles etc.
2. **Relations:**
   Relations represent the links between different objects. Relations can be unary(relations defined for a single term) and n-ary(relations defined for n terms).
   E.g., blue, round (unary); friends, siblings (binary); etc.
3. **Functions:**
   Functions map their input object to the output object using their underlying relation.
   Eg: father_of(), mother_of() etc.

Parts of First Order Logic

First-order logic in Artificial Intelligence comprises two main components, which are as follows.

1. **Syntax:**
   Syntax represents the rules to write expressions in First Order Logic in Artificial Intelligence.

2. **Semantics:**
   Semantics refers to the techniques that we use to evaluate an expression of First Order Logic in AI. These techniques use various known relations and facts of the respective environment to deduce the boolean value of the given First Order Logic expression.

We elaborate on the syntax of First Order Logic in AI in the following sections.

Syntax of First Order Logic

- **Syntax of First Order Logic** in Artificial Intelligence represents the rules of representing any natural language construct in terms of First Order Logic in AI.
- This involves the rules to describe any object and the relationships between different objects.
- These rules comprise rules for writing constants, variables, predicates, quantifiers, etc. In the following sections, we present various examples of the syntax of first-order logic in AI. These rules are further illustrated in the following sections.

Basic Elements of First-Order Logic

The following table furnishes the basic elements of First Order Logic in Artificial Intelligence.

Element	Example	Meaning
Constant	1, 2, A, John, Mumbai, cat, ….	Values that can not be changed
Variables	x, y, z, a, b, ….	Can take up any value and can also change
Predicates	Brother, Father, >, ….	Defines a relationship between its input terms
Function	sqrt, LeftLegOf, ….	Computes a defined relation of input term
Connectives	∧, ∨, ¬, ⇒, ⇔	Used to form complex sentences using atomic sentences
Equality	==	Relational operator that checks equality
Quantifier	∀, ∃	Imposes a quantity on the respective variable

Atomic Sentences

Atomic sentences are the most basic expressions of First Order Logic in AI. These sentences comprise a predicate followed by a set of terms inside a parenthesis. Formally stating, the structure of an atomic sentence looks like the following.

Predicate (term 1, term 2, term 3,...) Predicate (term 1, term 2, term 3,...)

One example of an atomic sentence in First Order Logic in Artificial Intelligence can be Sisters (Geeta , Seeta) Sisters (Geeta , Seeta). Another example involving a unary predicate can be Polygon (Rectangle) Polygon (Rectangle).

Complex Sentences

Complex sentences can be constructed by combining atomic sentences using connectives like AND (∧), OR (∨), NOT (¬), IMPLIES (⇒), IF AND ONLY IF (⇔) etc. Formally stating, if $c1$,$c2$,... represent connectives, a complex sentence in First Order Logic in AI can be defined as follows.

Predicate 1(term 1, term 2,...)$c1$ Predicate 2(term 1, term 2,...)$c2$...

An example of a complex sentence in First Order Logic in Artificial Intelligence can be *Sisters* (*Geeta*,*Seeta*) ∧ *Sisters* (*Geeta*,*Leela*). Another example using just one predicate can be ¬*Sisters*(*Geeta*,*Seeta*)

Any expression of First Order Logic in AI can be broken into two main components, namely subject and predicate. The subject is the main entity about which the expression describes. Predicate depicts the relation between different subjects. For example, in the expression, *Polygon*(*Rectangle*), which is a translation of "Rectangle is a polygon." in First Order Logic in Artificial Intelligence, "Rectangle" is the subject, and "Polygon" represents a predicate.

Quantifiers in First-Order Logic

Quantifiers in First Order Logic in AI, as the name suggests, are used to quantify any entity in a given environment. **Quantification** refers to the identification of the total number of an entity that is present in the environment and satisfies a given expression in First Order Logic in Artificial Intelligence. Quantifiers enable us to determine the range and scope of a variable in a logical expression. Two types of quantifiers are stated as follows.

1. Universal Quantifier
2. Existential Quantifier

Universal Quantifier

Universal Quantifier in First Order Logic in AI is a symbol in a logical expression that signifies that the given expression is true in its range for all instances of the concerned entity. It is represented by the symbol ∀∀ (an inverted A). If x is a variable, then ∀x is read as "For all x" or "For every x" or "For each x".

For example, let us take the sentence, "All cats like fish". Let us take a variable x which can take the value of "cat". Let us take a predicate cat (x) which is true if x is a cat. Similarly, let us take another predicate likes (x,y) which is true if x likes y. Therefore, using the universal quantifier ∀∀, we can write

∀x cat (x)⟹ likes (x, fish).

This expression is read as "For all x, if x is a cat, then x likes to fish".

Existential Quantifier

An existential Quantifier in First Order Logic in Artificial Intelligence is a symbol in a logical expression that signifies that the given expression is true in its range for at least one of the instances of the concerned entity. It is represented by the symbol ∃∃ (an inverted E).
If x is a variable, then ∃x is read as "There exists x" or "For some x" or "For at least one x".
For example, let us take the sentence, "Some students like ice cream". Let us take a variable x which can take the value of "student". Let us take a predicate s student (x), which is true if x is a student. Similarly, let us take another predicate likes (x,y), which is true if x likes y. Therefore, using the existential quantifier ∃∃, we can write
∃ student ∧ likes (ice-cream)∃x student (x)∧ likes (x, ice-cream)
This expression reads, "There exists some x such that x is a student and also likes ice cream".

Properties of Quantifiers

Quantifiers in First Order Logic in AI follow some properties as stated below.
In the universal quantifier,∀x∀y is equivalent to ∀y∀x.
In the existential quantifier, ∃x∃y is equivalent to ∃y∃x.
∃x∀y is not equivalent to ∀y∃x.
Quantifier Duality:
Each quantifier can be expressed using the other. E.g., ∀∀ x Predicate(x) is same as ¬ ∃∃ x ¬ Predicate(x).

Unification and lifting forward chaining and backward chaining

Forward Chaining and Backward Chaining are the two most important strategies in the field of Artificial Intelligence and lie in the Expert System Domain of AI.
Forward and Backward chaining is the strategies used by the Inference Engine in making the deductions.

Inference Engine:
Inference Engine is a component of the expert system that applies logical rules to the knowledge base to deduce new information. It interprets and evaluates the facts in the knowledge base in order to provide an answer.
A knowledgebase is a structured collection of facts about the system’s domain.

Forward Chaining:
Forward Chaining the Inference Engine goes through all the facts, conditions and derivations before deducing the outcome i.e When based on available data a decision is taken then the process is called as Forwarding chaining, It works from an initial state and reaches to the goal(final decision).

Example:
A
A -> B
B
_____
He is running.

If he is running, he sweats.
He is sweating.

Backward Chaining:
In this, the inference system knows the final decision or goal, this system starts from the goal and works backwards to determine what facts must be asserted so that the goal can be achieved, i.e it works from goal(final decision) and reaches the initial state.

Example:
B
A -> B
A
_____
He is sweating.
If he is running, he sweats.
He is running.

Difference between Forwarding Chaining and Backward Chaining:

	Forward Chaining	Backward Chaining
1.	When based on available data a decision is taken then the process is called as Forward chaining.	Backward chaining starts from the goal and works backward to determine what facts must be asserted so that the goal can be achieved.
2.	Forward chaining is known as data-driven technique because we reaches to the goal using the available data.	Backward chaining is known as goal-driven technique because we start from the goal and reaches the initial state in order to extract the facts.
3.	It is a bottom-up approach.	It is a top-down approach.
4.	It applies the Breadth-First Strategy.	It applies the Depth-First Strategy.
5.	Its goal is to get the conclusion.	Its goal is to get the possible facts or the required data.
6.	Slow as it has to use all the rules.	Fast as it has to use only a few rules.
7.	It operates in forward direction i.e it works from initial state to final decision.	It operates in backward direction i.e it works from goal to reach initial state.

8.	Forward chaining is used for the planning, monitoring, control, and interpretation application.	It is used in automated inference engines, theorem proofs, proof assistants and other artificial intelligence applications.